# Project Title: TURBO RUSH

**Course Name: Computer Graphics**

**GROUP NO: 04**

**SECTION: M**

**Supervised By**

**NOBORANJAN DEY**

**Submitted By:**

| Serial Number | Name | ID | Department |
|---|---|---|---|
| 21 | Monjurul Islam Nimon | 23-50004-1 | CSE |
| 22 | Prithiraj Modak | 23-50007-1 | CSE |
| 28 | Nishan Ahamed | 23-50040-1 | CSE |
| 27 | Md. Sadman Sakib | 23-50039-1 | CSE |
| 26 | Ariana Rahman Lazuk | 23-50019-1 | CSE |

**Date of submission:** February 04, 2025

## Introduction:

**➔What did we implement?**

We implemented a 2D car racing game using OpenGL (GLUT). The game includes the following features:

**Player-controlled car:** The user can control a green car that moves within predefined lanes.

**Opponent cars**: Randomly spawning red cars move down the screen to create a racing challenge.

**Obstacles:** Random obstacles (traffic cones, barrels, and roadblocks) appear to increase difficulty.

**Environment:** A road with lanes, lane dividers, zebra crossings, and footpath tiles.

**Collision detection**: If the player collides with an opponent car or obstacle, the game ends.

**Three difficulty levels:** The game offers Easy, Medium, and Hard levels to challenge players.

**Scoring system:** Players earn points based on how long they survive.

**➔Why did we implement it?**

**We implemented this game to:**

- Learn OpenGL (GLUT) programming:
- Implement fundamental 2D graphics rendering (cars, obstacles, background).
- Understand how to handle user input (e.g., moving the player car).
- Implement basic animations (moving cars and obstacles).

**Develop game logic:**

- Implement collision detection to detect crashes.
- Manage random spawning of cars and obstacles for gameplay variety.
- Keep track of game score and difficulty progression.

**Create an engaging and interactive game:**

- Introduce different game modes.
- Make the visuals with background colour changes and road elements.
- Implement lane-based mechanics to simulate a real racing experience.

- This project serves as a great learning experience for working with computer graphics, real-time rendering, and basic game development concepts.

**➔Significance of This Project:**

The 2D car racing game developed using OpenGL (GLUT) has both educational and practical significance, making it a valuable project in multiple ways:

**1. Educational Significance:**

- **<u>Graphics Programming & OpenGL Mastery:</u>**

1. Helps in understanding fundamental computer graphics concepts like rendering, transformations, and animations.
2. Introduces OpenGL pipeline and how to use GLUT for creating graphical applications.

- **<u>Game Development Basics:</u>**

1. Provides hands-on experience in developing game mechanics, including player control, opponent behaviour, and collision detection.
2. Enhances knowledge of game loop, event handling, and real-time rendering.

- **<u>Algorithm Implementation:</u>**

1. Requires randomization algorithms for spawning obstacles and opponent cars.
2. Uses collision detection techniques to determine crashes.
3. Levels & Difficulty Scaling:
4. Introduces concepts like difficulty balancing and how to modify game mechanics for different skill levels.

**2. Technical Significance:**

1. Understanding 2D Graphics Rendering: Helps in learning about coordinate systems, transformations (scaling, translation, rotation), and how to render dynamic objects.
2. Real-Time Interactions & Event Handling: Enhances programming skills in handling keyboard inputs, movement constraints, and real-time updates.
3. Game Optimization Techniques: Requires implementing efficient rendering techniques to maintain smooth gameplay performance.

**3. Practical & Real-World Applications:**

1. Foundation for Advanced Game Development: The concepts learned can be extended to 3D game development using OpenGL, Unity, or Unreal Engine.
2. Application in Simulation & AI: Can be extended for use in autonomous vehicle simulations, driving tests, or AI-based racing bots.
3. Problem-Solving & Creativity: Encourages students to think critically about game mechanics, user engagement, and interactive design.

**➔Target Population of This Project:**

The 2D Car Racing Game using OpenGL (GLUT) is designed for a wide range of users, including:

**1. Students & Beginners in Computer Graphics:**

- Computer Science and Engineering students learning OpenGL, game development, and computer graphics concepts.
- Beginners looking for hands-on experience in programming a simple yet interactive game.
- Those interested in game mechanics, event handling, and real-time rendering.

**2. Game Development Enthusiasts:**

- Aspiring game developers: who want to understand the basics of game loops, object rendering, collision detection, and difficulty scaling.
- Individuals interested in transitioning from 2D to 3D game development using OpenGL, Unity, or Unreal Engine.

**3. Educators:**

Teachers and professors who can use the project as an example or assignment for students in computer graphics, game programming, or simulation courses.

**4. Casual Gamers & Hobbyists:**

- People who enjoy simple, arcade-style racing games with increasing difficulty levels.
- Hobbyists who want to modify or customize the game by adding new features, such as power-ups or multiplayer modes.

**5. Developers & Open-Source Community:**

- Open-source contributors looking to enhance and expand existing game projects.
- Programmers who want to experiment with graphics rendering, animations, and user interaction in OpenGL.

## Tools:

### 1. GLUT (OpenGL Utility Toolkit):

- glutInit(): Initializes the GLUT library.
- glutInitDisplayMode(): Sets the display mode, such as double buffering and RGB color model.
- glutInitWindowSize(): Initializes the window size.
- glutCreateWindow(): Creates the window with a specified title.
- glutFullScreen(): Makes the window fullscreen.
- glutDisplayFunc(): Registers the display function that will render the scene.
- glutSpecialFunc(): Registers a special key press handler (for arrow keys, function keys, etc.).
- glutTimerFunc(): Sets a timer callback to update the scene at regular intervals (used for animation).
- glutMainLoop(): Starts the main loop for the GLUT application.
- GLUT_BITMAP_HELVETICA_18: Font used for displaying text in the window.
- glColor3f: Sets the color for drawing primitives.
- glBegin: Starts the drawing of primitives (such as triangles, quads).
- glVertex2f: Defines the vertices of the shapes.
- glEnd: Ends the drawing of primitives.
- glClear: Clears the color and depth buffers.
- glutSwapBuffers: Swaps the front and back buffers for double buffering.
- glutRasterPos2f: Specifies the position for rasterized text on the screen.

## Knowledge Applied Fields:

### 1. For Study Purpose (Educational Knowledge Fields Applied)

This project applies knowledge from several core fields, making it an excellent learning tool for students and individuals studying computer science and related disciplines:

**a. Computer Graphics:**

- OpenGL/GLUT Programming: Using OpenGL for rendering 2D graphics, including object creation, transformations, and animation.
- Rendering Techniques: Understanding how graphical elements are drawn and manipulated in a 2D environment.
- Colour Theory & Design: Implementing dynamic background transitions to simulate time-of-day changes.

**b. Programming Fundamentals:**

- C/C++ Programming: The game is coded in C or C++, with a focus on fundamental programming concepts like loops, conditionals, functions, and arrays.
- Event-Driven Programming: Handling user inputs, managing game state, and updating the display in real-time.
- Data Structures: Utilizing arrays, lists, and objects to store game elements such as cars, obstacles, and scores.

**c. Algorithms and Problem-Solving:**

- Collision Detection Algorithms: Detecting when objects (cars, obstacles) overlap using basic geometric collision detection techniques.
- Random Number Generation: Using randomization for the spawning of cars and obstacles to create a dynamic game environment.
- Game Logic: Developing algorithms to handle game rules, levels, scoring, and difficulty progression.

**d. User Interface Design:**

- Basic GUI: Creating simple graphical user interfaces for scoring, level transitions, and game status.
- Interaction Design: Allowing users to interact with the game (using keys to control the car).

**2. For Job Field (Knowledge Fields Applied in Professional Work):**

This project aligns with several skills relevant to careers in the tech industry, especially in areas of game development, graphics programming, and software engineering:

**a. Game Development:**

2D Game Design: Expertise in designing simple yet engaging 2D games, an essential skill for game developers working in casual or mobile games.

Game Mechanics: Knowledge of how to create and balance game systems, such as difficulty scaling and scoring mechanisms.

Game Engine Knowledge: While this project uses OpenGL/GLUT, the skills translate well to other game engines like Unity or Unreal Engine.

**b. Software Development:**

Software Engineering Principles: Understanding how to organize code into modular components, manage codebases, and design for scalability and efficiency.

Debugging & Optimization: Identifying and fixing performance issues and bugs, an important skill in game development and general software engineering.

**c. Computer Graphics & Visualization:**

Graphics Rendering Techniques: Knowledge of 2D rendering pipelines, and basic transformations (translation, scaling, rotation) within a graphical context.

Real-Time Rendering: Understanding how to render objects efficiently in real-time, which is critical for both games and graphical simulations.

**d. User Interface & User Experience Design:**

Interactive Design: Applying user interface principles to create an intuitive, engaging game experience.

UI/UX for Games: Knowledge of designing menus, scoreboards, and level transitions to create a cohesive user experience.

## Screenshots:

**Figure 01: Menu**

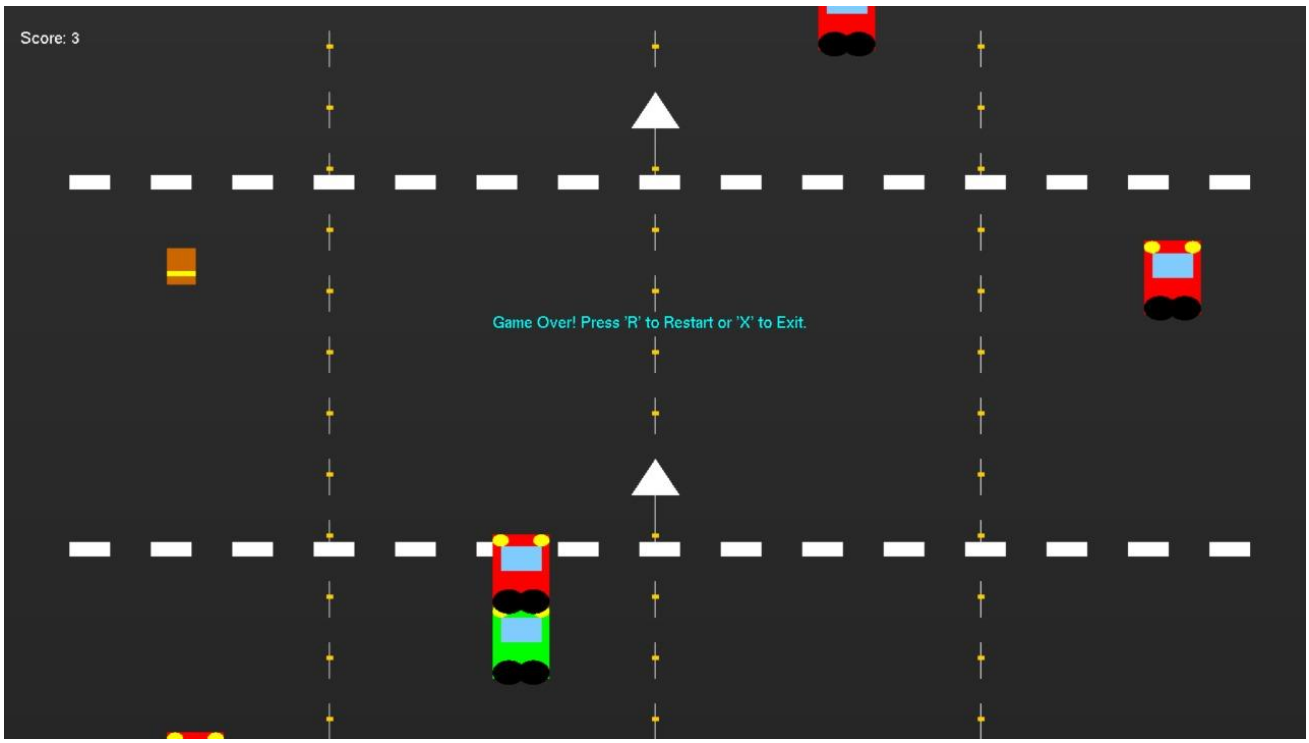


**Figure 02: Level 1 (EASY)**

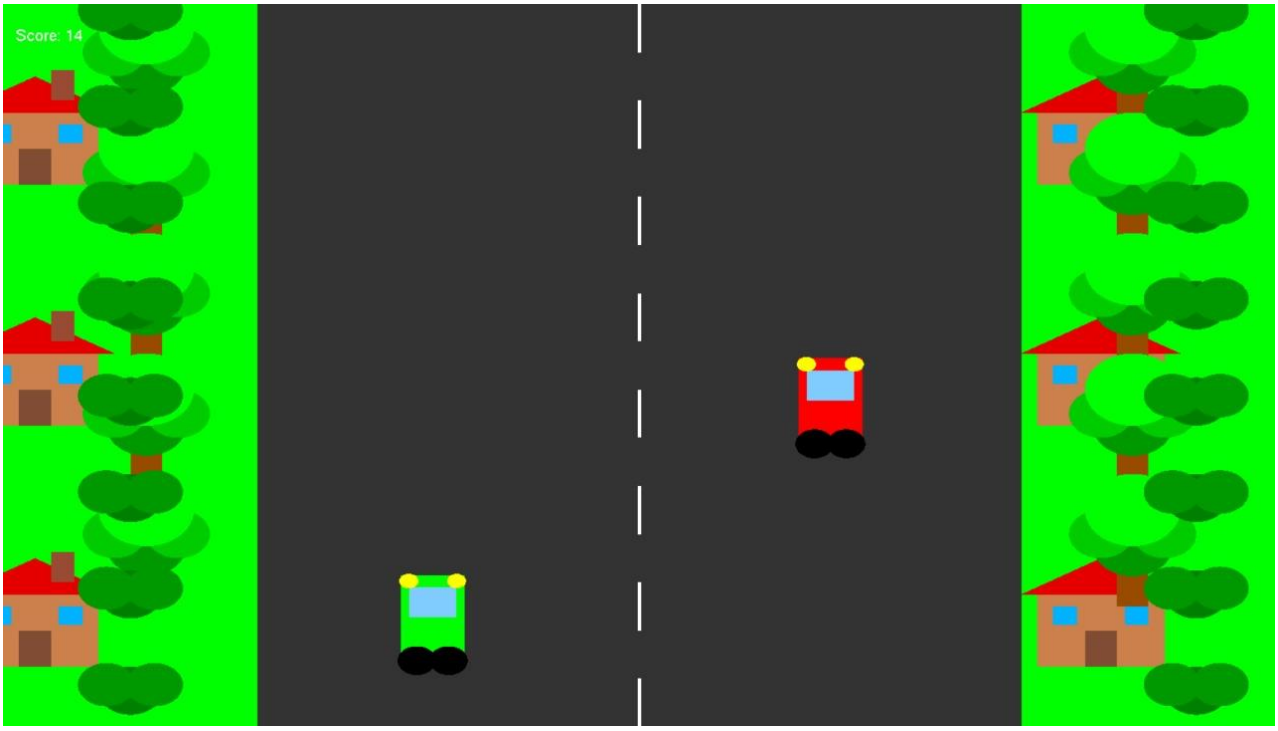


**Figure 03: Level 2 (MEDIUM)**

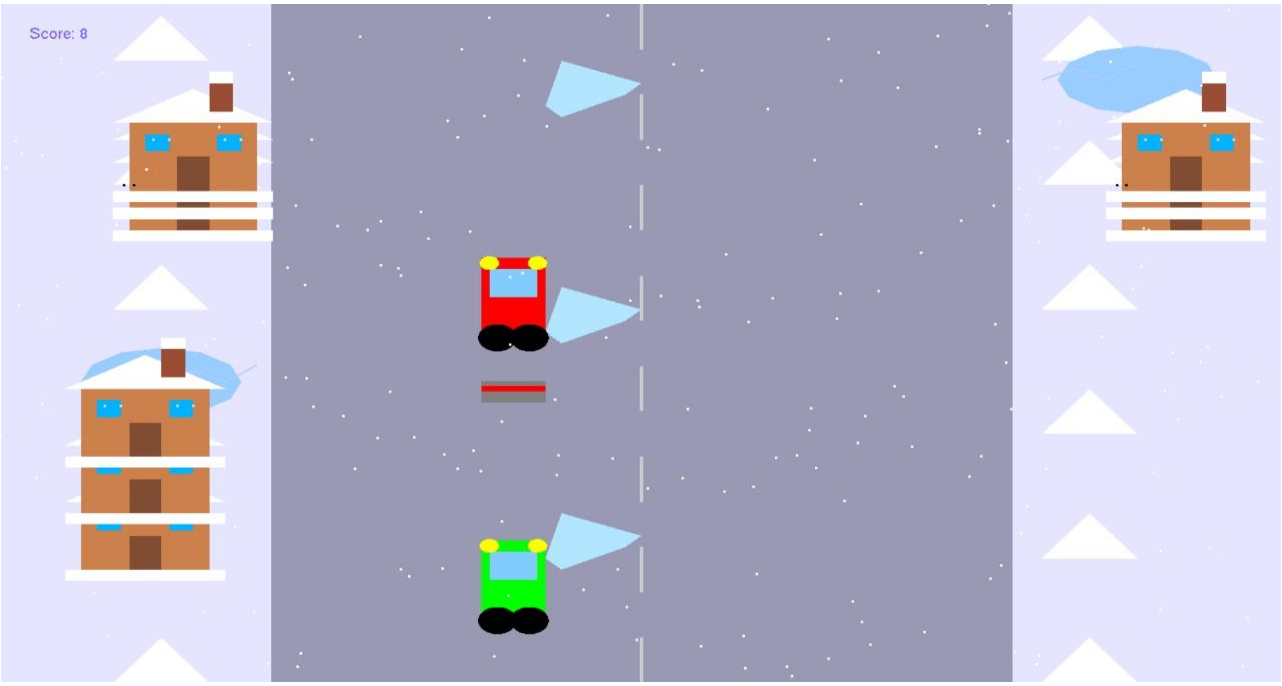


**Figure 04: Level 3 (Hard)**

## References:

https://github.com/monjurulnimon/Turbo-Rush